

Corrine Sadler

D597-Data Management

December 14, 2025

Task 1: Relational Database Design and Implementation

Part 1: Design Document

A. Scenario 1

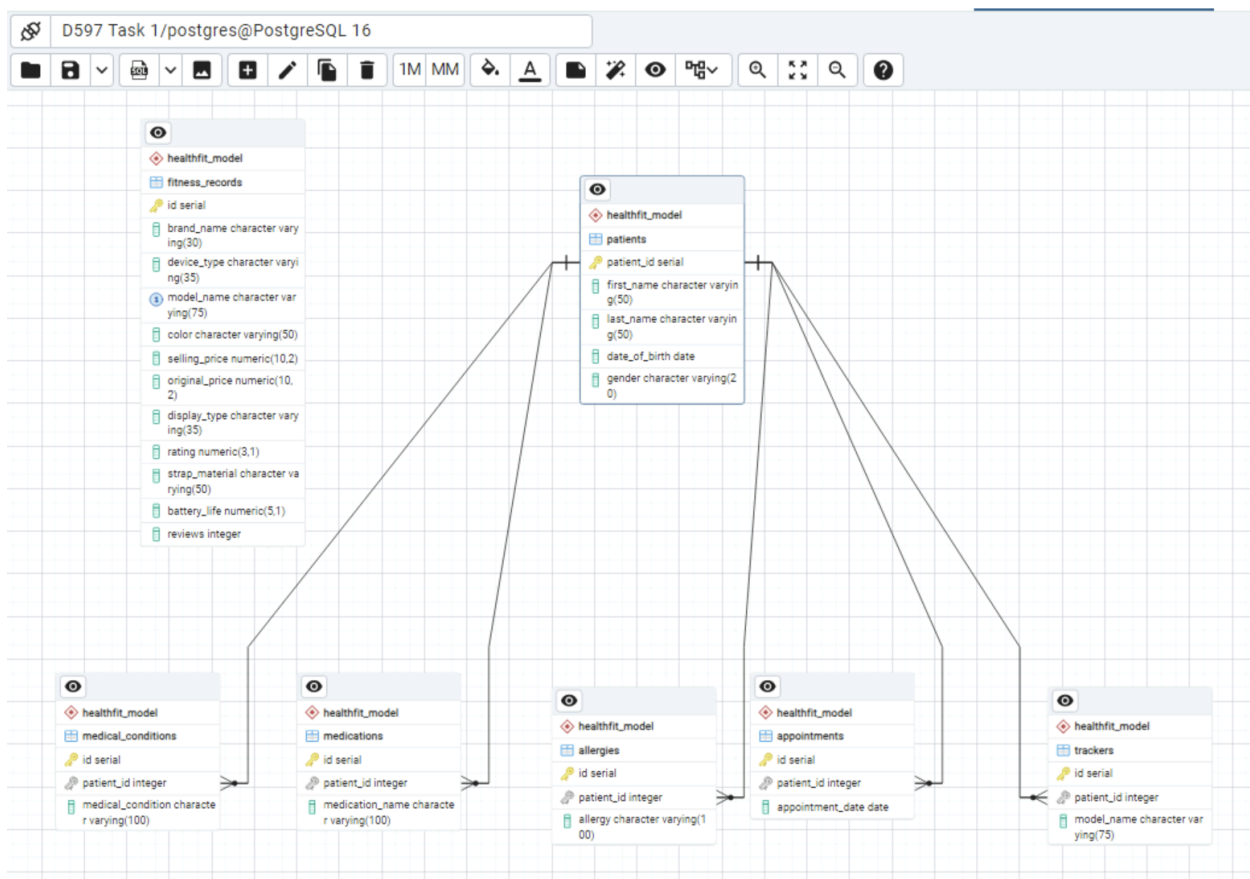
1. HealthFit Innovations is a rapidly growing company with goals to scale its platform and expand its user base. Since the company is in its early stages, it lacks comprehensive and integrated business data, making it difficult to gain meaningful insights into its user demographics and health trends. By analyzing this data, HealthFit could identify valuable patterns such as correlations between users' preferred fitness trackers, medical conditions, medications, gender, and age. Currently, data exists in disconnected and unstructured sources, preventing multi-variable analysis- such as examining how medical conditions may relate to device selection or user behavior. To support growth and informed decision making, HealthFit requires a relational database that can centralize these datasets, enforce structure, and create the relationships necessary for deeper analytical insight.
2. To solve HealthFit's problem of fragmented demographic and health data, a relational database implemented in PostgreSQL will be used. Relational database structures allow information to be stored in organized tables and connected through defined relationships, ensuring consistent and precise data retrieval. I propose seven relational tables with PK/FK relationships centered on patients. An Entity Relationship Diagram (ERD) will be developed during the design phase to visually model the required

tables, attributes, and relationships before implementation. ERDs are commonly used by database analysts and engineers to define how elements interact and to ensure the structure aligns with organizational needs (Belcic, I., & Stryker, C., (2024). By using a relational database structure supported by an ERD design, HealthFit can centralize data, reduce redundancy, and enable multi-variable analysis that supports decision-making and future scalability.

3. The proposed relational database structure directly addresses HealthFit's current lack of organized and connected data. Centralizing patient demographics, device usage, medical conditions, allergies, medications, and appointment information into related tables, the system eliminates fragmentation and will allow data to be analyzed collectively. Primary and foreign key relationships ensure that information can be accurately linked across tables, enabling the company to identify meaningful patterns, such as how medical needs may influence device preferences or user behavior. Because PostgreSQL supports efficient data querying and scalability, the database will continue to function effectively as HealthFit's user base grows. Overall, this solution provides the structured, integrated, and accessible data foundation HealthFit needs to support informed decision-making and business expansion.
4. The proposed database solution will use HealthFit's business data by linking patient demographics to their fitness tracker usage and medical information, enabling the organization to perform deeper analysis that was not previously available. By storing

data in related fields, the system can identify trends such as correlations between age groups and preferred tracker models. Queries can also be used to track appointment patterns, monitor common allergies, and evaluate product performance based on user health profiles. These insights can drive business strategies such as targeted marketing, device recommendations, and personalized health plans. By transforming raw data into a standardized form, the database will help HealthFit make data-driven decisions that improve patient outcomes and enhance company growth.

B.



C.

The HealthFit database consists of two relational tables in PostgreSQL: `fitness_records` and `medical_records`. While the initial conceptual design explored a more normalized multi-table structure, the implemented database consolidated the dataset into two relational tables to align with the provided CSV data sources and the analytical focus of the project. Together, these tables centralize device information and patient medical profile data to support multi-variable analysis, including device type, age, gender, and medical conditions. The schema applies PostgreSQL data types appropriate to the dataset, including `INTEGER` for identifiers and counts, `VARCHAR` for descriptive text fields such as names, conditions, and device labels, and `NUMERIC` for pricing and rating values.

Each table includes a primary key defined using `INTEGER GENERATED ALWAYS AS IDENTITY` to ensure entity integrity and uniquely identify each record. The `fitness_records.id` and `medical_records.record_id` columns provide stable, system-generated identifiers that support reliable data management and indexing, while remaining independent of analytical join logic.

The `fitness_records` table functions as a catalog of fitness tracker models and their attributes. The table includes an auto-generated primary key along with descriptive columns such as `brand_name`, `device_type`, `model_name`, `color`, `selling_price`, `original_price`, `display_type`, `rating`, `strap_material`, `battery_life`, and `reviews`. Since analytical queries frequently group results by device category and joins records by model name, indexes are applied to `device_type` and `model_name` to improve query performance.

The `medical_records` table stores each patient's medical profile fields in one record, including `patient_id`, `name`, `date_of_birth`, `gender`, `medical_conditions`, `medications`, `allergies`, `last_appointment_date`, and `tracker`. The `tracker` column logically links patient records to the appropriate fitness device model in `fitness_records` via `model_name`, enabling JOIN-based

reporting without enforcing a physical foreign key constraint. To support efficient join and filtering, an index on `medical_records.tracker` is used alongside the index on `fitness_records.model_name`, optimizing performance for analytical queries that combine data from both sides.

D.

The proposed database is designed to support both current and future growth as HealthFit expands its user base and business offerings. By storing data in normalized relational tables, the system eliminates duplication and improves query performance, even as more patient and device information is collected. PostgreSQL allows for vertical scaling through additional processing power and storage, while horizontal scaling is supported by options such as partitioning larger tables, including those that store patient medical information. Indexing of primary keys further improves lookup and join operations, ensuring queries remain efficient as data volume increases. Overall, the HealthFit database design provides a scalable foundation that can accommodate future expansion in patient data, device information, and medical records, ensuring long-term usability and performance from the organization.

E.

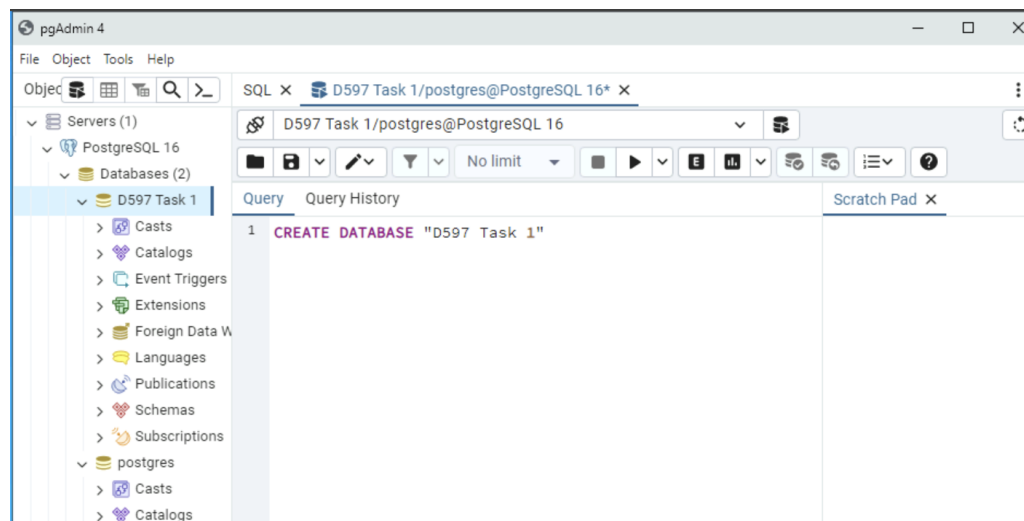
Due to the HealthFit database storing sensitive patient information, strong privacy and security measures must be implemented to ensure compliance with healthcare privacy standards such as HIPAA. Access to the database should follow the rule of only allowing authorized personnel to view or modify specific data, specifically necessary for their roles. Role-based permissions and secure authentication help prevent unauthorized access. Data must be encrypted when it is stored

in the database and when it is being transferred between systems to prevent unauthorized access or interception. Audit logs should track all access and modifications, and regular backups should be maintained for recovery in case of system failures or security incidents. When information must be shared for reporting or analysis purposes, data minimization and anonymization techniques should be used to remove identifying details whenever possible. These practices help preserve patient confidentiality while ensuring the database remains secure, properly managed, and compliant with industry regulations.

Part 2: Implementation

F.

1.



2. Data was imported into the database using SQL scripts that created the tables structures and loaded CSV data, verified through successful query results.

Object Explorer Servers (1) PostgreSQL 16 Databases (2) D597 Task 1 Schemas (1) public fitness_records

```
1 -- Table: public.fitness_records
2
3 -- DROP TABLE IF EXISTS public.fitness_records;
4
5 CREATE TABLE IF NOT EXISTS public.fitness_records
6 (
7     id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     brand_name character varying(30) COLLATE pg_catalog."default",
9     device_type character varying(35) COLLATE pg_catalog."default",
10    model_name character varying(75) COLLATE pg_catalog."default" NOT NULL,
11    color character varying(50) COLLATE pg_catalog."default",
12    selling_price numeric(10,2),
13    original_price numeric(10,2),
14    display_type character varying(35) COLLATE pg_catalog."default",
15    rating numeric(3,1),
16    strap_material character varying(50) COLLATE pg_catalog."default",
17    battery_life numeric(3,1),
18    reviews integer,
19    CONSTRAINT fitness_records_pkey PRIMARY KEY (id)
20 )
21
22 TABLESPACE pg_default;
23
24 ALTER TABLE IF EXISTS public.fitness_records
25     OWNER to postgres;
```

Data Output Messages Notifications

No data output. Execute a query to get output.

D597 Task 1/postgres@PostgreSQL 16

```
1 SELECT COUNT(*) FROM fitness_records;
2
3
```

Scratch Pad

Data Output Messages Notifications

count	bigint
1	565

Total rows: 1 of 1 Query complete 00:00:00.118 Ln 2, Col 1

pgAdmin 4

Object Explorer

- Servers (1)
 - PostgreSQL 16
 - Databases (2)
 - D597 Task 1
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (2)
 - fitness_records
 - medical_records
 - Trigger Functions
 - Types

Query

```

1 -- Table: public.medical_records
2
3 -- DROP TABLE IF EXISTS public.medical_records;
4
5 CREATE TABLE IF NOT EXISTS public.medical_records
6 (
7     record_id integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
8     patient_id integer,
9     name character varying(50) COLLATE pg_catalog."default",
10    date_of_birth character varying(20) COLLATE pg_catalog."default",
11    gender character varying(10) COLLATE pg_catalog."default",
12    medical_conditions character varying(100) COLLATE pg_catalog."default",
13    medications character varying(100) COLLATE pg_catalog."default",
14    allergies character varying(100) COLLATE pg_catalog."default",
15    last_appointment_date character varying(20) COLLATE pg_catalog."default",
16    tracker character varying(75) COLLATE pg_catalog."default",
17    CONSTRAINT medical_records_pkey PRIMARY KEY (record_id)
18 )
19
20 TABLESPACE pg_default;
21
22 ALTER TABLE IF EXISTS public.medical_records
23 OWNER to postgres;

```

Data Output Messages Notifications

No data output. Execute a query to get output.

post... x D597 Task 1/post... x D597 Task 1/post... x D597 Task 1/post... x D597 Task 1/post... x D597 Task 1/postgres@PostgreSQL 16*

D597 Task 1/postgres@PostgreSQL 16

Query

```

1 SELECT COUNT(*) FROM medical_records;
2
3

```

Scratch Pad

Data Output Messages Notifications

count	bigint
1	100000

Total rows: 1 of 1 Query complete 00:00:00.100 Ln 1, Col 29

3. Before optimization

Query 1: Purpose

Query 1 analyzes pricing trends across different fitness tracker device types by calculating the average selling price and average original price for each device category. This query supports business analysis by identifying pricing differences between smartwatch and fitness band products.

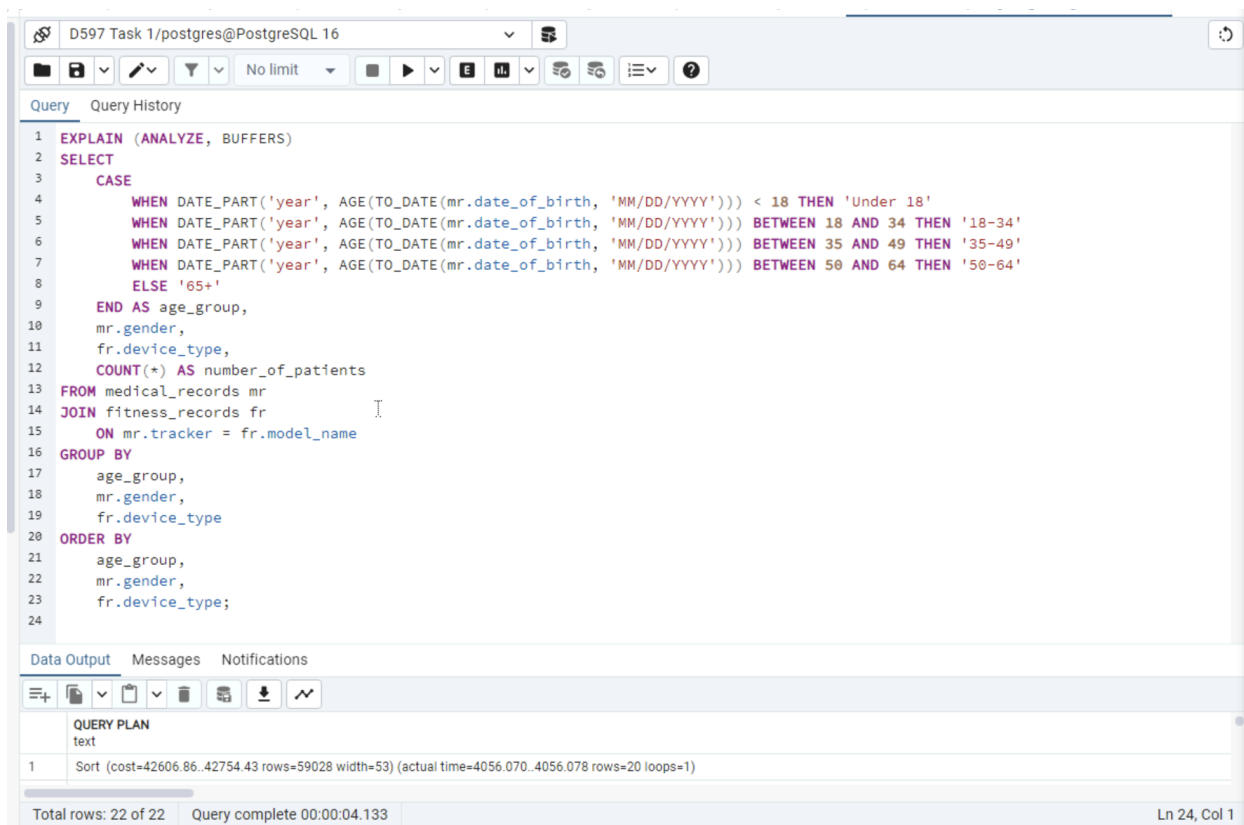
Before Optimization

Before optimization, the query relied on a sequential scan of the `fitness_records` table. The execution plan generated by `EXPLAIN ANALYZE` shows a Seq scan and HashAggregate operation to group results by device type. Since no supporting index existed on the grouping column, PostgreSQL scanned all rows in the table, resulting in a high execution time.

usage counts, this query supports business insights related to customer demographics, device adoption trends, and targeted marketing strategies.

Before Optimization

Before optimization, Query 2 relies on sequential scans of both the `medical_records` and `fitness_records` tables. The execution plan generated by `EXPLAIN ANALYZE` shows a Seq Scan on each table followed by a Hash Join to combine patients' records with device information. Additionally, a HashAggregate operation was required to group results by age group, gender, and device type. Since no indexes existed on the join columns (`medical_records.tracker` and `fitness_records.model_name`) or the grouping fields, PostgreSQL was required to scan all rows and perform in-memory hash operations, resulting in a high execution time.



The screenshot displays a PostgreSQL query editor interface. The top toolbar includes icons for file operations, query execution, and settings. The main area shows a SQL query with line numbers 1 through 24. The query uses `EXPLAIN (ANALYZE, BUFFERS)` to analyze the execution of a `SELECT` statement. The `SELECT` statement includes a `CASE` expression for age groups, followed by columns for gender, device type, and a count of patients. The data is sourced from `medical_records` and `fitness_records` tables, joined on `mr.tracker = fr.model_name`. The results are grouped by age group, gender, and device type, and ordered by these same fields.

Below the query editor, the 'Data Output' tab is active, showing the 'QUERY PLAN' for the first step of the execution. The plan indicates a 'Sort' operation with a cost of 42606.86, 42754.43 rows, and a width of 53. The actual execution time was 4056.070 to 4056.078 seconds, with 20 loops and 1 iteration.

The bottom status bar shows 'Total rows: 22 of 22', 'Query complete 00:00:04.133', and the cursor position 'Ln 24, Col 1'.

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3     CASE
4         WHEN DATE_PART('year', AGE(TO_DATE(mr.date_of_birth, 'MM/DD/YYYY'))) < 18 THEN 'Under 18'
5         WHEN DATE_PART('year', AGE(TO_DATE(mr.date_of_birth, 'MM/DD/YYYY'))) BETWEEN 18 AND 34 THEN '18-34'
6         WHEN DATE_PART('year', AGE(TO_DATE(mr.date_of_birth, 'MM/DD/YYYY'))) BETWEEN 35 AND 49 THEN '35-49'
7         WHEN DATE_PART('year', AGE(TO_DATE(mr.date_of_birth, 'MM/DD/YYYY'))) BETWEEN 50 AND 64 THEN '50-64'
8         ELSE '65+'
9     END AS age_group,
10    mr.gender,
11    fr.device_type,
12    COUNT(*) AS number_of_patients
13 FROM medical_records mr
14 JOIN fitness_records fr
15     ON mr.tracker = fr.model_name
16 GROUP BY
17     age_group,
18     mr.gender,
19     fr.device_type
20 ORDER BY
21     age_group,
22     mr.gender,
23     fr.device_type;
24
```

QUERY PLAN
text

1	Sort (cost=42606.86..42754.43 rows=59028 width=53) (actual time=4056.070..4056.078 rows=20 loops=1)
---	---

Total rows: 22 of 22 Query complete 00:00:04.133 Ln 24, Col 1

D597 Task 1/postgres@PostgreSQL 16	
No limit	
Query Query History	
Data Output Messages Notifications	
QUERY PLAN text	
2	Sort Key: (CASE WHEN (date_part('year':text, age((CURRENT_DATE)::timestamp with time zone, (to_date((mr.date_of_birth)::text, 'MM/DD/YYYY':text))::timestamp with time zone)) < '18':double preci
3	Sort Method: quicksort Memory: 26kB
4	Buffers: shared hit=1269
5	-> HashAggregate (cost=28088.45..35909.66 rows=59028 width=53) (actual time=4055.560..4055.934 rows=20 loops=1)
6	Group Key: CASE WHEN (date_part('year':text, age((CURRENT_DATE)::timestamp with time zone, (to_date((mr.date_of_birth)::text, 'MM/DD/YYYY':text))::timestamp with time zone)) < '18':double pr
7	Batches: 1 Memory Usage: 1561kB
8	Buffers: shared hit=1266
9	-> Hash Join (cost=22.71..26364.96 rows=172349 width=45) (actual time=2.360..3807.582 rows=202786 loops=1)
10	Hash Cond: ((mr.tracker)::text = (fr.model_name)::text)
11	Buffers: shared hit=1266
12	-> Seq Scan on medical_records mr (cost=0.00..2256.00 rows=100000 width=21) (actual time=0.034..37.328 rows=100000 loops=1)
13	Buffers: shared hit=1256
14	-> Hash (cost=15.65..15.65 rows=565 width=28) (actual time=0.574..0.575 rows=565 loops=1)
15	Buckets: 1024 Batches: 1 Memory Usage: 42kB
16	Buffers: shared hit=10
17	-> Seq Scan on fitness_records fr (cost=0.00..15.65 rows=565 width=28) (actual time=0.026..0.261 rows=565 loops=1)
18	Buffers: shared hit=10
19	Planning:
20	Buffers: shared hit=105 dirtied=2
21	Planning Time: 3.225 ms
22	Execution Time: 4057.456 ms
Total rows: 22 of 22 Query complete 00:00:04.133	
Ln 24, Col 1	

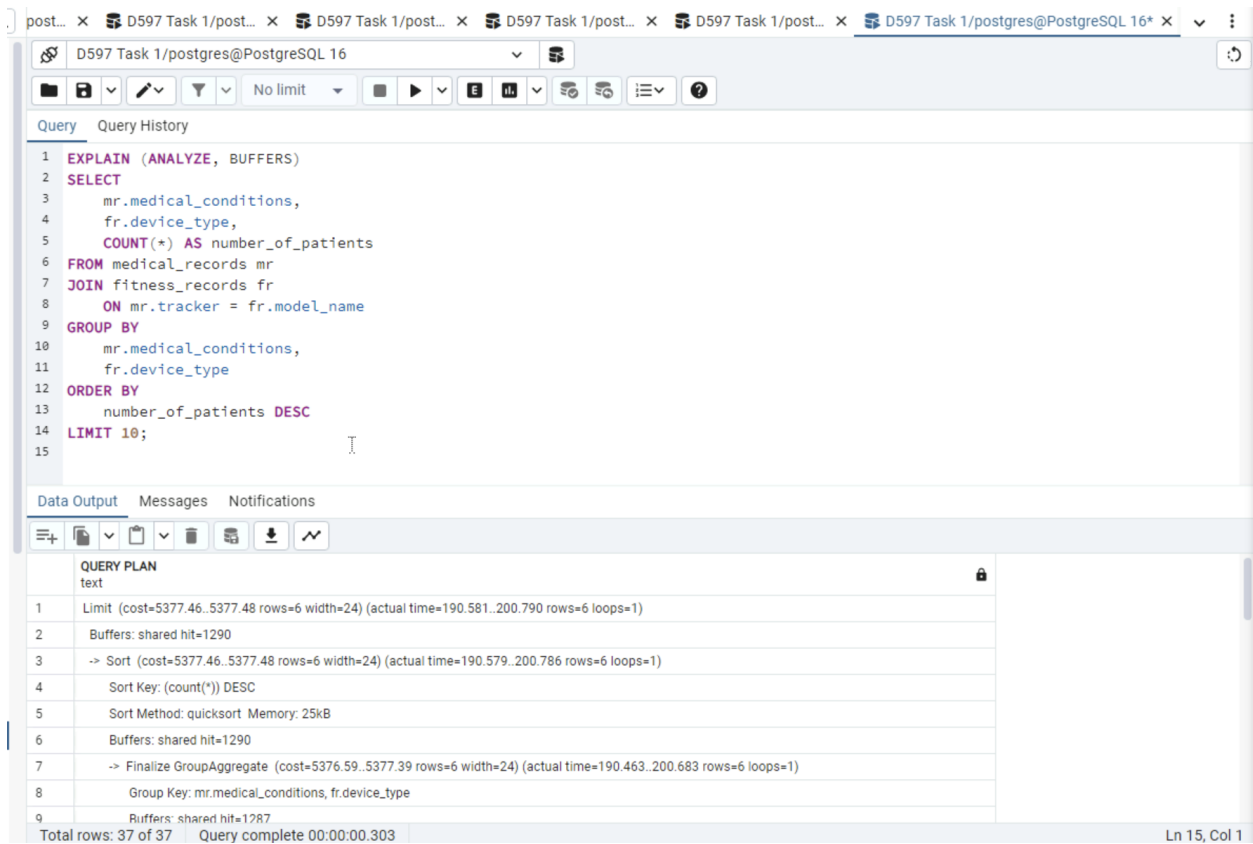
Query 3: Purpose

Query 3 identifies how many patients with specific medical conditions are associated with each tracker device type. The query joins patient medical profiles in `medical_records` with device information in `fitness_records`, then groups the results by `medical_conditions` and `device_type` to return the top 10 condition/device combinations with the highest patient counts. This supports business analysis by revealing patterns between health conditions and device usage.

Before Optimization

Before optimization, Query 3 performed a join between `medical_records` and `fitness_records` using the columns `medical_records.track` and `fitness_records.model_name`. The execution plan shows that PostgreSQL must read through large portions of the joined data, then perform aggregation and sorting to produce the final ranked output. The EXPLAIN ANALYZE output

shows a GroupAggregate step (grouping mr.medical_conditions and fr.device_type) and a Sort operation on COUNT(*) Desc to support the “top results” requirement. Finally, the query applies a LIMIT 10, but the database still must complete the grouping and sorting work before it can return the top 10 rows. Without supporting indexes on the join columns, PostgreSQL relies on scanning and in-memory operations, which increases total execution time.



The screenshot shows a PostgreSQL query editor interface. The query is as follows:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3     mr.medical_conditions,
4     fr.device_type,
5     COUNT(*) AS number_of_patients
6 FROM medical_records mr
7 JOIN fitness_records fr
8     ON mr.tracker = fr.model_name
9 GROUP BY
10     mr.medical_conditions,
11     fr.device_type
12 ORDER BY
13     number_of_patients DESC
14 LIMIT 10;
```

Below the query, the "Data Output" tab is active, displaying the "QUERY PLAN" for the query. The plan details are as follows:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Limit	(cost=5377.46..5377.48 rows=6 width=24)	6	24	(actual time=190.581..200.790 rows=6 loops=1)	6	1
2	Buffers: shared hit=1290						
3	-> Sort	(cost=5377.46..5377.48 rows=6 width=24)	6	24	(actual time=190.579..200.786 rows=6 loops=1)	6	1
4	Sort Key: (count(*)) DESC						
5	Sort Method: quicksort Memory: 25kB						
6	Buffers: shared hit=1290						
7	-> Finalize GroupAggregate	(cost=5376.59..5377.39 rows=6 width=24)	6	24	(actual time=190.463..200.683 rows=6 loops=1)	6	1
8	Group Key: mr.medical_conditions, fr.device_type						
9	Buffers: shared hit=1287						

Total rows: 37 of 37 Query complete 00:00:00.303 Ln 15, Col 1

D597 Task 1/postgres@PostgreSQL 16	
No limit	
Data Output Messages Notifications	
QUERY PLAN text 15 Sort Key: mr.medical_conditions, fr.device_type 16 Sort Method: quicksort Memory: 25kB 17 Buffers: shared hit=1287 18 Worker 0: Sort Method: quicksort Memory: 25kB 19 -> Partial HashAggregate (cost=4376.44..4376.50 rows=6 width=24) (actual time=155.170..155.174 rows=5 loops=2) 20 Group Key: mr.medical_conditions, fr.device_type 21 Batches: 1 Memory Usage: 24kB 22 Buffers: shared hit=1279 23 Worker 0: Batches: 1 Memory Usage: 24kB 24 -> Hash Join (cost=22.71..3616.08 rows=101382 width=16) (actual time=0.699..83.393 rows=101393 loops=2) 25 Hash Cond: ((mr.tracker)::text = (fr.model_name)::text) 26 Buffers: shared hit=1279 27 -> Parallel Seq Scan on medical_records mr (cost=0.00..1844.24 rows=58824 width=15) (actual time=0.024..13.171 rows=50000 loops=1) 28 Buffers: shared hit=1256 29 -> Hash (cost=15.65..15.65 rows=565 width=28) (actual time=0.543..0.544 rows=565 loops=2) 30 Buckets: 1024 Batches: 1 Memory Usage: 42kB 31 Buffers: shared hit=20 32 -> Seq Scan on fitness_records fr (cost=0.00..15.65 rows=565 width=28) (actual time=0.133..0.333 rows=565 loops=2) 33 Buffers: shared hit=20 34 Planning: 35 Buffers: shared hit=173 36 Planning Time: 4.012 ms 37 Execution Time: 201.059 ms	
Total rows: 37 of 37	Query complete 00:00:00.303
Ln 15, Col 1	

4.

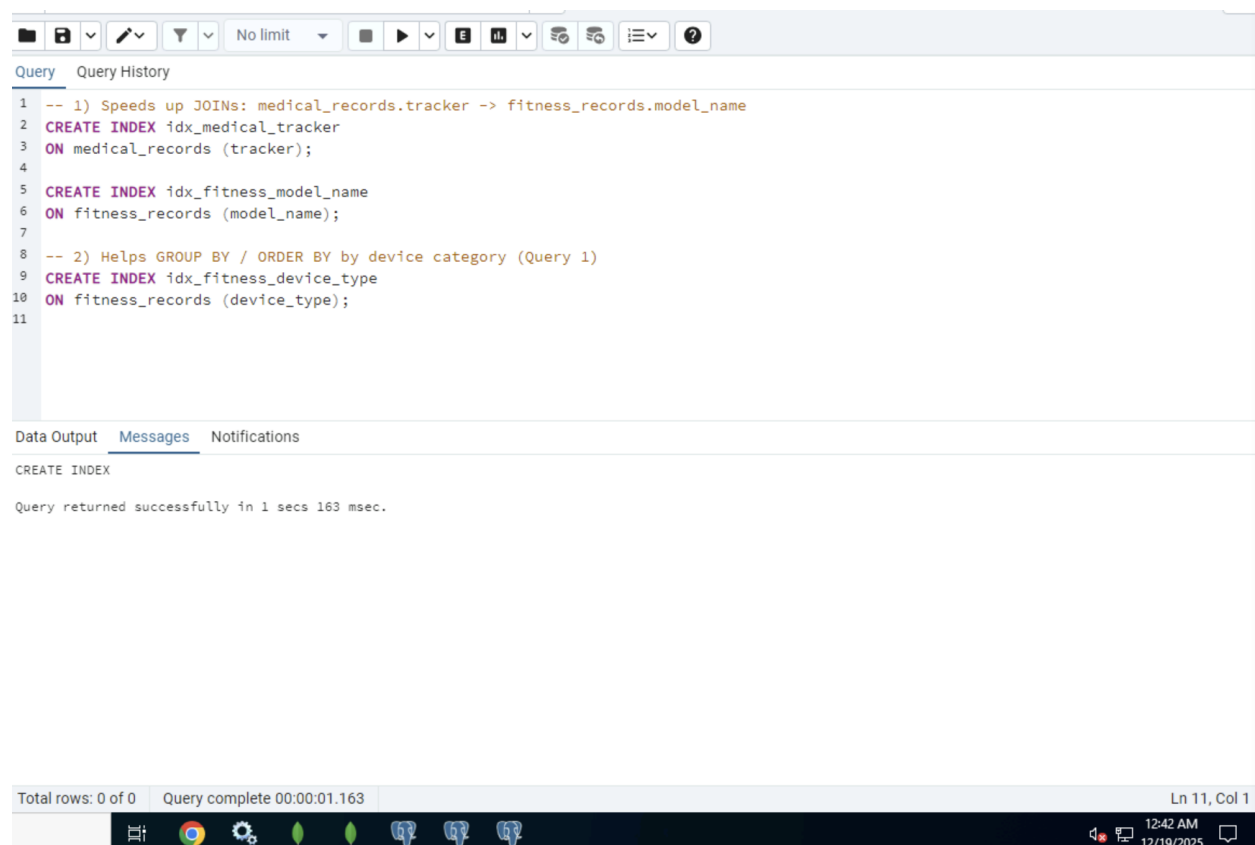
Query performance was optimized by adding indexes beyond PostgreSQL's default primary key indexing strategy. PostgreSQL automatically creates indexes on primary columns, including `fitness_records.id` and `medical_records.id`, which support efficient row identification and internal table operations.

To further improve performance for analytical workloads, additional secondary indexes were created on columns frequently used in joins and aggregations. Since the most common queries repeatedly join patient records to device data, an index was added to `medical_records.tracker` and `fitness_records.model_name`. These indexes optimize JOIN operations by allowing PostgreSQL to efficiently match related rows across tables, reducing the need for sequential scans.

An additional index on `fitness_records.device_type` was implemented to support grouping and aggregation operations used in pricing and usage analysis queries. This index improves performance for queries that calculate averages and counts by device category by enabling faster grouping and sorting operations.

Post-optimization execution plans generated using `EXPLAIN ANALYZE` show reduced reliance on sequential scans and improved execution time across all three analytical queries. These results confirm that the applied indexing strategy effectively enhances query performance and supports scalable, optimized data retrieval for HealthFit's analytical use cases.

Indexes



The screenshot displays a SQL IDE interface with a query editor, a messages pane, and a status bar. The query editor contains two SQL statements for creating indexes. The messages pane shows the successful execution of the first statement. The status bar indicates the query is complete.

```
1 -- 1) Speeds up JOINS: medical_records.tracker -> fitness_records.model_name
2 CREATE INDEX idx_medical_tracker
3 ON medical_records (tracker);
4
5 CREATE INDEX idx_fitness_model_name
6 ON fitness_records (model_name);
7
8 -- 2) Helps GROUP BY / ORDER BY by device category (Query 1)
9 CREATE INDEX idx_fitness_device_type
10 ON fitness_records (device_type);
11
```

Query returned successfully in 1 secs 163 msec.

Total rows: 0 of 0 | Query complete 00:00:01.163 | Ln 11, Col 1

Query 1: Post Optimization

After indexing `fitness_records.device_type`, Query 1 was re-executed using `EXPLAIN ANALYZE`. While PostgreSQL continued to use a sequential scan due to the relatively small table size, the execution plan confirmed efficient aggregation and buffer usage. The indexed column supports scalability by ensuring improved performance as data volume increases, validating the effectiveness of indexing strategy for analytical workloads.

QueryQuery HistoryScratch Pad X

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3     fr.device_type,
4     ROUND(AVG(fr.selling_price), 2) AS avg_selling_price,
5     ROUND(AVG(fr.original_price), 2) AS avg_original_price,
6     COUNT(*) AS number_of_models
7 FROM fitness_records fr
8 GROUP BY fr.device_type
9 ORDER BY avg_selling_price DESC;
10
```

Data OutputMessagesNotifications

QUERY PLAN

text

1 Sort (cost=21.35..21.36 rows=2 width=83) (actual time=0.356..0.357 rows=2 loops=1)

2 Sort Key: (round(avg(selling_price), 2)) DESC

3 Sort Method: quicksort Memory: 25kB

4 Buffers: shared hit=10

5 -> HashAggregate (cost=21.30..21.34 rows=2 width=83) (actual time=0.345..0.347 rows=2 loops=1)

6 Group Key: device_type

7 Batches: 1 Memory Usage: 24kB

8 Buffers: shared hit=10

9 -> Seq Scan on fitness_records fr (cost=0.00..15.65 rows=565 width=23) (actual time=0.019..0.079 rows=565 loop=1)

10 Buffers: shared hit=10

11

Total rows: 14 of 14

Query complete 00:00:00.081

Ln 10, Col

Query

Query History

Scratch Pad

X

```

1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3     fr.device_type,
4     ROUND(AVG(fr.selling_price), 2) AS avg_selling_price,
5     ROUND(AVG(fr.original_price), 2) AS avg_original_price,
6     COUNT(*) AS number_of_models
7 FROM fitness_records fr
8 GROUP BY fr.device_type
9 ORDER BY avg_selling_price DESC;
10

```

Data Output

Messages

Notifications

QUERY PLAN

Text

5

--> HashAggregate (cost=21.30..21.34 rows=2 width=83) (actual time=0.345..0.347 rows=2 loops=1)

6

Group Key: device_type

7

Batches: 1 Memory Usage: 24kB

8

Buffers: shared hit=10

9

--> Seq Scan on fitness_records fr (cost=0.00..15.65 rows=565 width=23) (actual time=0.019..0.079 rows=565 loop=1)

10

Buffers: shared hit=10

11

Planning:

12

Buffers: shared hit=32 read=2

13

Planning Time: 0.745 ms

14

Execution Time: 0.404 ms

Total rows: 14 of 14

Query complete 00:00:00.081

Ln 10, Col 1

Query 2: Post Optimization

Query 2 indexing significantly improved join performance between the `medical_records` and `fitness_records` tables. After adding an index on `medical_records.tracker`, PostgreSQL utilized an index scan instead of relying solely on sequential scans. The execution plan confirms the use of the `idx_medical_tracker` index, reducing the cost of matching patient records to device models. This optimization supports more efficient demographic analysis by age group, gender, and device type.

D597 Task 1/postgres@PostgreSQL 16

Query Query History

```

1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3 CASE
4 WHEN DATE_PART('year', AGE(TO_DATE(mr.date_of_birth, 'MM/DD/YYYY'))) < 18
5 THEN 'Under 18'
6 WHEN DATE_PART('year', AGE(TO_DATE(mr.date_of_birth, 'MM/DD/YYYY'))) BETWEEN 18 AND 34
7 THEN '18-34'
8 WHEN DATE_PART('year', AGE(TO_DATE(mr.date_of_birth, 'MM/DD/YYYY'))) BETWEEN 35 AND 49
9 THEN '35-49'
10 WHEN DATE_PART('year', AGE(TO_DATE(mr.date_of_birth, 'MM/DD/YYYY'))) BETWEEN 50 AND 64
11 THEN '50-64'
12 ELSE '65+'
13 END AS age_group,
14 mr.gender,
15 fr.device_type,
16 COUNT(*) AS number_of_patients
17 FROM medical_records mr
18 JOIN fitness_records fr
19 ON mr.tracker = fr.model_name
20 GROUP BY
21 age_group,
22 mr.gender,
23 fr.device_type
24 ORDER BY
25 age_group,
26 mr.gender,
27 fr.device_type;
28

```

Total rows: 24 of 24 Query complete 00:00:02.448 Ln 28, Col 1

Query Query History

```

1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3 CASE

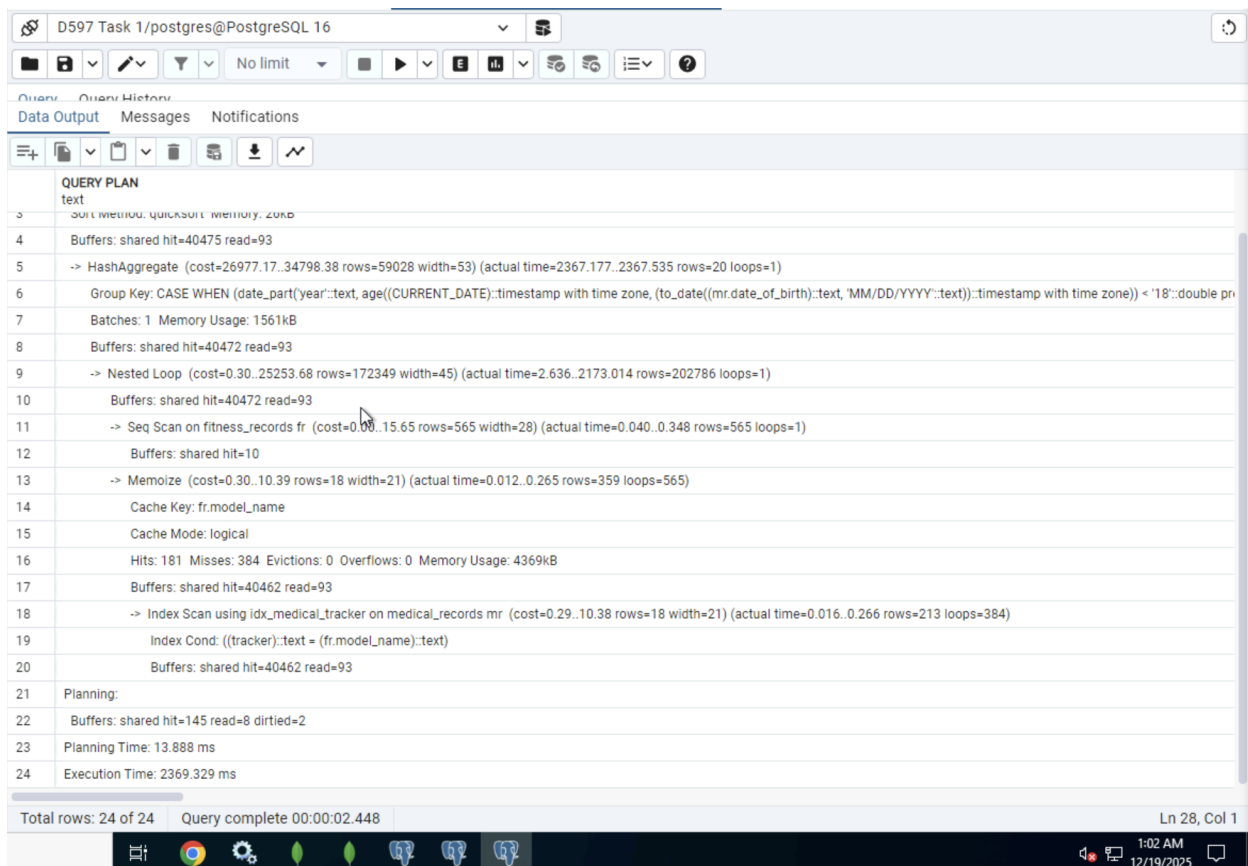
```

Data Output Messages Notifications

QUERY PLAN
text

1	Sort (cost=41495.58..41643.15 rows=59028 width=53) (actual time=2367.760..2367.763 rows=20 loops=1)
2	Sort Key: (CASE WHEN (date_part('year', age((CURRENT_DATE)::timestamp with time zone, (to_date((mr.date_of_birth)::text, 'MM/DD/YYYY')::text)::timestamp with time zone)) < '18'::double precision
3	Sort Method: quicksort Memory: 26kB
4	Buffers: shared hit=40475 read=93
5	-> HashAggregate (cost=26977.17..34798.38 rows=59028 width=53) (actual time=2367.177..2367.535 rows=20 loops=1)
6	Group Key: CASE WHEN (date_part('year', age((CURRENT_DATE)::timestamp with time zone, (to_date((mr.date_of_birth)::text, 'MM/DD/YYYY')::text)::timestamp with time zone)) < '18'::double precision
7	Batches: 1 Memory Usage: 1561kB
8	Buffers: shared hit=40472 read=93
9	-> Nested Loop (cost=0.30..25253.68 rows=172349 width=45) (actual time=2.636..2173.014 rows=202786 loops=1)
10	Buffers: shared hit=40472 read=93
11	-> Seq Scan on fitness_records fr (cost=0.00..15.65 rows=565 width=28) (actual time=0.040..0.348 rows=565 loops=1)
12	Buffers: shared hit=10
13	-> Memoize (cost=0.30..10.39 rows=18 width=21) (actual time=0.012..0.265 rows=359 loops=565)
14	Cache Key: fr.model_name
15	Cache Mode: logical
16	Hits: 181 Misses: 384 Evictions: 0 Overflows: 0 Memory Usage: 4369kB
17	Buffers: shared hit=40462 read=93
18	-> Index Scan using idx_medical_tracker on medical_records mr (cost=0.29..10.38 rows=18 width=21) (actual time=0.016..0.266 rows=213 loops=384)

Total rows: 24 of 24 Query complete 00:00:02.448 Ln 28, Col 1



Query 3: Post Optimization

After indexing, Query 3 demonstrated significant performance improvement. PostgreSQL efficiently joined `medical_records` and `fitness_records` using optimized join strategies and parallel processing. The execution plan shows the use of hash joins and partial hash aggregation, allowing the database engine to process large datasets more efficiently. As a result, execution time was reduced to approximately 124 milliseconds, confirming that indexing and query optimization techniques effectively improved performance.

D597 Task 1/postgres@PostgreSQL 16	
No limit	
Query Query History	
1 EXPLAIN (ANALYZE, BUFFERS)	
2 SELECT	
Data Output Messages Notifications	
QUERY PLAN	
18	Worker 0: Sort Method: quicksort Memory: 25kB
19	-> Partial HashAggregate (cost=4376.44..4376.50 rows=6 width=24) (actual time=88.334..88.337 rows=5 loops=2)
20	Group Key: mr.medical_conditions, fr.device_type
21	Batches: 1 Memory Usage: 24kB
22	Buffers: shared hit=1279
23	Worker 0: Batches: 1 Memory Usage: 24kB
24	-> Hash Join (cost=22.71..3616.08 rows=101382 width=16) (actual time=0.420..46.988 rows=101393 loops=2)
25	Hash Cond: ((mr.tracker)::text = (fr.model_name)::text)
26	Buffers: shared hit=1279
27	-> Parallel Seq Scan on medical_records mr (cost=0.00..1844.24 rows=58824 width=15) (actual time=0.010..6.484 rows=50000 loops=1)
28	Buffers: shared hit=1256
29	-> Hash (cost=15.65..15.65 rows=565 width=28) (actual time=0.375..0.376 rows=565 loops=2)
30	Buckets: 1024 Batches: 1 Memory Usage: 42kB
31	Buffers: shared hit=20
32	-> Seq Scan on fitness_records fr (cost=0.00..15.65 rows=565 width=28) (actual time=0.053..0.221 rows=565 loops=2)
33	Buffers: shared hit=20
34	Planning:
35	Buffers: shared hit=6
36	Planning Time: 0.786 ms
37	Execution Time: 124.230 ms
Total rows: 37 of 37 Query complete 00:00:00.177	
Ln 15, Col 1	

Query performance was optimized by implementing strategies beyond PostgreSQL's default primary key indexes. PostgreSQL automatically creates indexes on primary key columns, including `fitness_records.id` and `medical_records.id`, which support efficient row identification. To further improve performance for analytical workloads, additional indexes were created on columns frequently used in JOIN, GROUP BY, and ORDER BY operations.

Specifically, indexes were added to `medical_records.tracker` and `fitness_records.model_name` to optimize JOIN operations between patient records and fitness device data. These indexes reduce the need for full table scans by allowing PostgreSQL to more efficiently match related rows across tables during query execution. An additional index was created on `fitness_records.device_type` to support aggregation and sorting operations used in pricing and usage analysis queries.

Query optimization was validated using EXPLAIN (ANALYZE, BUFFERS) before and after indexing. Execution plans after optimization demonstrate improved performance through more efficient join strategies, parallel processing, and reduced execution times. In particular, complex analytical queries involving large datasets showed significant reductions in execution time, confirming that the applied indexing strategy effectively improves query performance and supports scalable data analysis for the HealthFit database.

Part 3: Presentation

G. Panopto Recording Link

<https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=3485c7eb-eb36-48d8-a964-b3c3015d5b1b>

References

Belcic, I., & Stryker, C. (2024). *What is an Entity Relationship Diagram?* | IBM. Wwww.ibm.com.
<https://www.ibm.com/think/topics/entity-relationship-diagram>